AWS CLOUD ENGINEERING & OPERATIONS

Monthly Grocery – Vercel & Supabase to AWS Migration Blueprint

Step-by-step technical handbook to migrate **PostgreSQL Database**, **Express Backend API**, **Next.js Web Admin**, and **S3 Media Storage** directly to Amazon Web Services.

ORIGIN STACK	TARGET CLOUD	CORE SERVICES	DOWNTIME PLAN
Vercel + Supabase	**AWS (ap-south-1 Mumbai)**	**EC2, RDS, S3, Amplify**	**< 2 Minutes (DNS Switch)**

📍 **CURRENT PRODUCTION STACK**

- **Web Admin:** Vercel (Next.js 16)
- **Backend API:** Vercel Serverless / Local
- **Database:** Supabase PostgreSQL
- **Files:** Supabase Storage / Local

➔

🚀 **TARGET AWS CLOUD STACK**

- **Web Admin:** AWS Amplify (Next.js 16)
- **Backend API:** AWS EC2 (Ubuntu 24.04 + PM2)
- **Database:** Amazon RDS PostgreSQL (gp3)
- **Files:** Amazon S3 + CloudFront CDN

BACKEND SERVER
AWS EC2 (t4g.small)
Ubuntu 24.04 • PM2 24/7

DATABASE
Amazon RDS (Postgres)
20GB gp3 SSD • Auto Backup

MEDIA STORAGE
Amazon S3 Bucket
Product images & invoices

WEB ADMIN
AWS Amplify / EC2
Auto CI/CD • Free SSL

Step 1: AWS Foundation & Security Firewall Setup

Establish Asia Pacific (Mumbai) region and configure security firewall groups

AWS CORE

1.1 Set AWS Region

Set AWS Console region to **Asia Pacific (Mumbai)** `ap-south-1` for lowest latency across India (~20ms).

1.2 Create Security Group

Go to **EC2 > Security Groups > Create**: Name it `monthlygrocery-ec2-sg` inside your Default VPC.

RULE TYPE	PORT	SOURCE IP	OPERATIONAL PURPOSE
SSH	`22`	`My IP` (or 0.0.0.0/0)	Encrypted terminal access for server maintenance
HTTP	`80`	`0.0.0.0/0` (Public)	Inbound web traffic & Let's Encrypt SSL verification
HTTPS	`443`	`0.0.0.0/0` (Public)	Encrypted REST API & Web Admin traffic
PostgreSQL	`5432`	`monthlygrocery-ec2-sg`	Database access restricted exclusively to backend server

Step 2: Migrate Supabase Database ➔ Amazon RDS PostgreSQL

Export all tables, sequences, orders, and products to Amazon RDS

DATABASE

2.1 Create RDS PostgreSQL Instance

RDS > Create Database: Standard Create > PostgreSQL 16 > Free Tier > Identifier: `monthlygrocery-prod-db` > Master user: `postgres` > Class: `db.t4g.micro` > Storage: 20GB gp3 > Attach Security Group: `monthlygrocery-ec2-sg`.

2.2 Export Database from Supabase

Run `pg_dump` from your terminal to generate a complete SQL export including schema, relations, and row data without permission locks.

Terminal (Local Machine) – 1. Export from Supabase

```
pg_dump -h db.xlnebedclqcmgfbfqkbm.supabase.co -U postgres -p 5432 -d postgres --clean --if-exists --no-owner --no-privileges -f monthlygrocery_backup.sql
```

Terminal (Local Machine) – 2. Restore into Amazon RDS

```
psql -h monthlygrocery-prod-db.c3xxx.ap-south-1.rds.amazonaws.com -U postgres -d postgres -f monthlygrocery_backup.sql
```

✅ **Verification:** Connect via `psql -h <rds-endpoint> -U postgres` and run `\dt` to verify tables: `profiles`, `shops`, `products`, `orders`, `coupons`, `delivery_slots`.

Step 3: Amazon S3 Bucket & IAM Access Keys

Object storage for product photos, store banners, user avatars & Excel sheets

STORAGE

3.1 Create S3 Bucket

S3 > Create bucket: Name: `monthlygrocery-prod-assets-2026` > Region: `ap-south-1` > ACLs enabled > Uncheck *"Block all public access"* so product images are viewable by customer apps.

3.2 Create IAM Programmatic User

IAM > Users > Create User: `monthlygrocery-backend-uploader` > Attach Policy: `AmazonS3FullAccess` > Create Access Key > Copy **Access Key ID** and **Secret Key**.

S3 CORS Policy (S3 > Bucket > Permissions > CORS)

```
[
  {
    "AllowedHeaders": ["*"],
    "AllowedMethods": ["GET", "PUT", "POST", "HEAD"],
    "AllowedOrigins": ["*"],
    "ExposeHeaders": []
  }
]
```

Step 4: Launch AWS EC2 Virtual Server & Deploy Backend API

Host Express REST API server 24/7 with PM2 daemon process management

COMPUTE

4.1 Launch EC2 Instance

EC2 > Launch Instance: Ubuntu Server 24.04 LTS > Type: `t4g.small` (or `t3.small`) > Create Key Pair: `monthlygrocery-key.pem` > Select Security Group: `monthlygrocery-ec2-sg` > Storage: 30GB gp3.

4.2 Configure Server Environment

SSH into your instance and install Node.js 20 LTS, Git, build tools, Nginx web server, and PM2 process manager globally.

EC2 Terminal – Install Stack & Clone Backend

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git build-essential nginx
sudo npm install -g pm2 typescript

sudo mkdir -p /var/www && sudo chown -R ubuntu:ubuntu /var/www
cd /var/www && git clone https://github.com/your-org/MonthlyGrocery.git
cd MonthlyGrocery/express-backend && npm install && npm run build
```

/var/www/MonthlyGrocery/express-backend/.env

```
PORT=8001
NODE_ENV=production
DATABASE_URL=postgresql://postgres:PASSWORD@monthlygrocery-prod-db.c3xxx.ap-south-1.rds.amazonaws.com:5432/postgres
JWT_SECRET=super-secure-production-jwt-random-key-2026-mg
DEV_OTP_BYPASS=false
AWS_ACCESS_KEY_ID=YOUR_IAM_ACCESS_KEY
AWS_SECRET_ACCESS_KEY=YOUR_IAM_SECRET_KEY
AWS_REGION=ap-south-1
AWS_S3_BUCKET=monthlygrocery-prod-assets-2026
MIN_ORDER_LIMIT=3000
```

EC2 Terminal – Start Backend API with PM2 Daemon

```
pm2 start dist/server.js --name "monthlygrocery-api"
pm2 startup
pm2 save
```

Step 5: Nginx Reverse Proxy & Free Let's Encrypt SSL

Securely route api.monthlygrocery.in (Port 443 HTTPS) to internal Port 8001

NGINX & SSL

EC2 Terminal – Enable Nginx Site & Issue Free Automated SSL

```
# 1. Write Nginx config (/etc/nginx/sites-available/monthlygrocery-api)
sudo tee /etc/nginx/sites-available/monthlygrocery-api << 'EOF'
server {
    listen 80;
    server_name api.monthlygrocery.in;
    location / {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        client_max_body_size 50M;
    }
}
EOF

sudo ln -s /etc/nginx/sites-available/monthlygrocery-api /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl restart nginx

# 2. Issue automated SSL cert using Certbot
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.monthlygrocery.in
```

Step 6: Deploy Next.js 16 Web Admin on AWS

Deploy the Super Admin portal via AWS Amplify Hosting or same EC2

FRONTEND

- **Option A (Recommended): AWS Amplify**

Amplify > Create App > Connect GitHub: App root: `web-admin` > Env variable: `NEXT_PUBLIC_API_URL=https://api.monthlygrocery.in/api` > Domain: `admin.monthlygrocery.in` (SSL auto-provisioned).

- **Option B: Same EC2 Instance (Port 3000)**

`cd /var/www/MonthlyGrocery/web-admin && npm install && npm run build` > Run with PM2: `pm2 start npm --name "web-admin" -- start -- -p 3000`.

Step 7: Update Mobile Apps API Endpoints

Point Customer App & Merchant App from development/Vercel to AWS

MOBILE APPS

mobile-app/src/config/api.ts & merchant-app/src/config/api.ts

```
// Production AWS Live API Endpoint
export const API_BASE_URL = __DEV__
  ? 'http://10.0.2.2:8001/api'           // Android emulator local debug
  : 'https://api.monthlygrocery.in/api';  // AWS Live Production API
```

Step 8: Domain DNS Cutover & Live Verification

Point custom domains to AWS and verify end-to-end grocery workflows

LIVE LAUNCH

RECORD	HOST / NAME	TARGET / DESTINATION VALUE	TTL
A Record	`api` (api.monthlygrocery.in)	`YOUR_EC2_PUBLIC_IP`	Auto / 300s
CNAME	`admin` (admin.monthlygrocery.in)	`d1xxx.amplifyapp.com` (or EC2 IP)	Auto / 300s

- **Customer Flow Test**

1. Open Customer App > 2. OTP Login > 3. Pin delivery address on Google Map > 4. Add items to Monthly Basket > 5. Select Slot > 6. Place COD order.

- **Merchant & Admin Test**

1. Open Merchant App > Check live order card > Update status to "Out for Delivery" > 2. Open Web Admin > Verify live order analytics and stock counts.

Migration Complete: Your Monthly Grocery ecosystem is now 100% running natively on AWS with high availability and scalability!